**TIP** From the same Packages page, you can also delete the published image or the entire image repository (called a *package* in GitHub) by going to the Package Settings via the link in the sidebar.

That concludes this section. Now that you know the main features of container images, how to create them, and how to publish them, let's dive deeper into packaging Spring Boot applications as images.

## *6.2 Packaging Spring Boot applications as container images*

In the previous chapters, we built the Catalog Service application, featuring a REST API and database integration. In this section, as an intermediate step before deploying it to Kubernetes, we'll build an image to run Catalog Service as a container on Docker.

First I'll review some aspects you should consider when packaging a Spring Boot application as a container image. Then I'll show you how to do that with a Dockerfile and with Cloud Native Buildpacks.

### *6.2.1 Preparing Spring Boot for containerization*

Packaging a Spring Boot application as a container image means that the application will run in an isolated context, including computational resources and network. Two main questions may arise from this isolation:

- How can you reach the application through the network?
- How can you make it interact with other containers?

We'll look at those two issues next.